Document number:	P1248R0
Date:	2018-10-07
Audience:	Library Evolution Working Group
Reply-to:	*Tomasz Kamiński <tomaszkam at gmail dot com>*

Fixing `Relations`

1. Introduction

This paper proposes changing the `Relation` and related concepts, to achieve following goals:

- *zero overhead*: uses of range version of standard algorithm shall not be less effective than hand written code, especially it shall not be less effective than STL1 counterparts,
- *generality*: concepts should be usable both in case of algorithms and containers, they should map to the underlying mathematical concepts and avoid imposing additional requirement,
- *soundness*: the syntactic requirements of the algorithm should allow to check (in runtime) if supplied object satisfies associated semantic requirement.

The secondary goal is to simplify migration to the rangified version of algorithms, however reaching of this goal should not undermine above properties.

This paper argues that the currently specified `Relation` concept does not achieve above goal, due the additional requirement of `CommonReference` for types of parameters of callable. As consequence we propose to relax this concept (and derived `StrictWeakOrdering` concept) by removing above requirement. [Note: This affects only requirements on comparators used with STL algorithms, and are not affecting relational operators (`<`, `==`, ...).]

The need for applying this change, was recognized with the inception of the current concept design - similar change is discussed as alternative design in Appendix D.2 Cross-type Concepts of A Concept Design for the STL (The Palo Alto report) paper.

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

Family of the standard algorithms related to sorting and operations on sorted sequence (`sort`, `lower_bound`, `set_intersection`, ...) are accepting an functor, that is representing a *weak ordering*. Examples of such ordering, includes relation resulting from a composition of the *key function* that extracts certain part of the object, and the *strong ordering* relation over extracted part.

To illustrate lets us consider following `NameLess` functor, that is comparing object of `Employee` by comparing only `name` member:

```
struct NameLess
{
   bool operator()(Employee const& lhs, Employee const& rhs) const
   { return lhs.name < rhs.name; }

   bool operator()(Employee const& lhs, std::string_view rhs) const
   { return lhs.name < rhs; }

   bool operator()(std::string_view lhs, Employee const& rhs) const
   { return lhs < rhs.name; }

   bool operator()(std::string_view lhs, std::string_view rhs) const
   { return lhs < rhs; }
};
```

Having above functor, we may sort a vector `ve` of the `Employee` object by name:

```
std::sort(ve.begin(), ve.end(), NameLess{});
```

Now that code can be upgraded to use Ranges:

```
std::ranges::sort(ve.begin(), ve.end(), NameLess{});
std::ranges::sort(ve, NameLess{});
```

Finally we have an option to provide our *key function* and *strong ordering* separately:

```
std::ranges::sort(ve, std::less<>{}, &Employee::name);
```

Once the vector is sorted, we can use one of the other algorithms to find all employee's with given name (i.e. find *equivalence class* for given value):

```
std::equal_range(ve.begin(), ve.end(), "Kowalski"sv, NameLess{});
```

However, if we want to migrate the above code to Ranges, by stating:

```
std::ranges::equal_range(ve.begin(), ve.end(), "Kowalski"sv, NameLess{});
std::ranges::equal_range(ve, "Kowalski"sv, NameLess{});
```

The above invocations will not compile, and we will be forced to rewrite my functor into separate projection and comparison:

```
std::ranges::equal_range(ve, "Kowalski"sv, std::less<>{}, &Employee::name);
```

The above compilation problem is caused by the fact that for the relation to satisfy `StrictWeakOrdering<NameLess, Employee, std::string_view>`, such functor needs to satisfy `Relation<NameLess, Employee, std::string_view>`, which requires that the types `Employee` and `std::string_view` has a common reference (`CommonReference<Employee, std::string_view>`).

In situation when *weak ordering* is defined in terms of the composition of the *key function* and *strong ordering*, the type returned by *key function* may be unrelated to the actual object type, as we are usually returning a member of that object. This means that the user is forced to rewrite they comparators into separate projection and homogeneous functor. This puts an limitation on the implementation freedom for the users, and have negative consequences on expressiveness and runtime performance of the code.

3.1. Inefficient abstraction

To illustrate the effect of above limitation, let us consider the following problem: We have a `vector<Itinerary>` representing round-trip flights (to specific destination and back), and we want to find ones that are having departures on specific dates.

Firstly we defined the required comparators, `LegDepartureDateLess` compares departure date on leg, while `ItneraryDepartureDatesLess` compares departures of all legs:

```
struct LegDepartureDateLess
{
   bool operator()(Leg const& lhs, Leg const& rhs) const
   { return lhs.departureDate < rhs.departureDate; }

   bool operator()(Leg const& lhs, std::chrono::local_days const& rhs) const
   { return lhs.departureDate < rhs; }

   bool operator()(std::chrono::local_days const& lhs, Leg const& rhs) const
   { return lhs < rhs.departureDate; }

   bool operator()(std::chrono::local_days const& lhs, std::chrono::local_days const& rhs) const
   { return lhs < rhs; }
};

struct ItineraryDepartureDatesLess
{
  bool operator()(Itinerary const& lhs, Itinerary const& rhs) const
  { return less_impl(lhs.legs, rhs.legs); }

  bool operator()(std::vector<std::chrono::local_days> const& lhs, Itinerary const& rhs) const
  { return less_impl(lhs, rhs.legs); }

  bool operator()(Itinerary const& lhs, std::vector<std::chrono::local_days> const& rhs) const
  { return less_impl(lhs.legs, rhs); }

  bool operator()(std::vector<std::chrono::local_days> const& lhs, std::vector<std::chrono::local_days> const& rhs) const
  { return less_impl(lhs, rhs); }

private:
  template<typename Sequence1, typename Sequence2>
  bool less_impl(Sequence1 const& lhs, Sequence2 const& rhs) const
  {
    return std::lexicographical_compare(lhs.begin(), lhs.end(),
                                        rhs.begin(), rhs.end(),
                                        LegDepartureDateLess{});
  }
};
```

Given above comparison functions, we can sort the vector `vi` of `Itinerary`:

```
std::sort(vi.begin(), vi.end(), ItineraryDepartureDatesLess{});
```

After that we can search for specific set of departure dates (given as vector `vd`) in the logarithmic time:

```
std::equal_range(vi.begin(), vi.end(), vd, ItineraryDepartureDatesLess{});
```

If we want to implement the same functionality using the ranges, we can mechanically transform the code sorting elements:

```
std::ranges::sort(vi, ItineraryDepartureDatesLess{});
```

However such manual transformation will not work if we want to search itineraries departing on specific dates (`std::vector<std::chrono::local_days> vd`):

```
std::equal_range(vi, vd, ItineraryDepartureDatesLess{});
```

will not compile, as (unsuprisingly) there is no common reference between the `Itinerary` and `std::vector<std::chrono::local_days>` — it will be ambiguous if `std::vector<std::chrono::local_days>` meant to represent departure or arrival dates of either all legs or all flights constituting given itinerary.

To fix above code, we need to split our comparator into separate projection and relation. In the first attempt we may define projection as follows:

```
auto const toDepartureDates = [](Itinerary const& i) -> std::vector<std::chrono::local_days>
{
   std::vector<std::chrono::local_days> result(i.legs.size());
   std::ranges::transform(i.legs, result.begin(), &Leg::departureDate);
   return result;
}
```

For the comparator, we just need to invoke `lexicographical_compare` giving following code:

```
std::equal_range(vi, vd,
                 [](auto const& lhs, auto const& rhs) { return std::ranges::lexicographical_compare(lhs, rhs); },
                 toDepartureDates);
```

The above code has a major drawback - for each comparison of elements (that is limited by $O(nlog(n))$) a temporary vector is created, leading to degradation of the code performance comparing to STL1 example.

This problem can be mitigated by returning a transform view from the `toDepartureDates` function, to reduce the cost of temporary production:

```
auto const toDepartureDatesView = [](Itinerary const& i) { return i.legs | ranges::view::transform(&Leg::departureDate); };
```

Above solution does not eliminate the problems caused by common reference requirement, just pushes is one level down.

```
std::equal_range(vi, vd,
                 [](auto const& lhs, auto const& rhs) { return std::ranges::lexicographical_compare(lhs, rhs); },
                 toDepartureDatesView);
```

In the above code, provided lambda needs to satisfy `StrictWeakOrdering` on `std::vector<std::chrono::local_days>` and `TransformView` (result of `toDepartureDateView`), which means that common reference between these two types needs to exists. This happens to be true for this specific case, as all proposed views are convertible to containers, however if we modify the example to use `std::span<std::chrono::local_days>` instead of `std::vector<std::chrono::local_days>` the code will again fail to compile, as there is no common reference for different views - this specialization also cannot be defined by user.

As presented above, the implementation options limitation imposed by requiring a `CommonReference` for orderings, is negatively impacting the performance in the resulting code. Furthermore, this problem will not be fully addressed by the introduction of the views from P0896R2: The One Ranges Proposal in C++ standard, as resulting code may fail victim of the same problem — code fails to compile due lack of `CommonReference` for views. In authors opinion, this means that current design of Ranges are breaking zero-overhead principle, which is backbone of C++ language - the user is able to implement same functionality in more efficient manner, by simply using older version of algorithms, that was not imposing such limitation.

3.2. Mathematical soundness

One of the arguments for the introduction of the `CommonReference` requirement, is that it is **required** to make `StrictWeakOrdering` mathematically sound. The definition of soundness is a bit subjective, but to counteract the argument, the author proposes following approach: The concept is considered to be sound, if it:

- semantic requirements are consistent with the semantic requirements of the underlying mathematical definition,
- syntactic requirements allow above semantic requirement to be verified in code (if necessary at runtime).

In our case we are interested in the definition of the *weak ordering* over some domain *X*. The mathematical definition of *weak ordering* requires following axioms to be true:

- *irreflexivity*: for all `x` in *X*: `!r(x, x)`
- *asymmetry*: for all `x`, `y` in *X*: `r(x, y)` => `!r(y, x)`
- *transitivity*: for all `x`, `y`, `z` in *X*: `r(x, y) && r(y, z)` => `r(x, z)`
- *transitivity of incomparability*: for all `x`, `y`, `z` in *X*: `(!r(x, y) && !r(y, x)) && (!r(y, z) && !r(z, y))` => `(!r(x, z) && !r(z, x))`

The last requirement is necessary to show that the function `eq(x, y)` equivalent to `!r(x, y) && !r(y, x)` is an equivalence relation, i.e. the relation is *reflexive* (guaranteed by *irreflexivity* of `r`), *symmetric* (from definition), and *transitive* (imposed directly by *transitivity of incomparability* for `r`). This guarantees that for any `x`, `y` in the *X* only one of following is true: `r(x, y)`, `eq(x, y)`, `r(y, x)`, where `eq` is equality relation. That means that `r` also satisfies definition of *weak ordering* presented in section "4.2 Total and Weak Orderings" of "Elements of Programming" by Alexander Stepanov and Paul McJones.

To verify if given functor `f` is strict weak ordering over an domain consisting of objects of type `T` and of type `U` (note that domain is just a set of elements, which may be of different type), we will require following invocation to be well-formed: `f(t, u)`, `f(u, t)`, `f(t, t)`, `f(u, u)`, where `t` is object of type `T` and `u` of type `U`.

In summary, neither the semantic requirement of the *weak ordering* nor the syntactic requirement imposed by them require existence of the common reference (common type) for the objects in the domain. As a consequence the definition of `StrictWeakOrdering` that does not impose `CommonReference` is equally sound as current definition.

Furthermore, introduction of such additional requirement is limiting the generality of the underlying algorithms (as can work with any *weak ordering*) which is a step against ideas of generic programming.

3.3. Tailored for algorithms

The current definition of the `StrictWeakOrdering`, was coined in the context of its usage with newly proposed range version of standard algorithms, that accept separate relation and projection arguments. This reasoning, misses other standard library components, that also accepts comparators. This is especially important in case of ordered containers (`set`, `map`) that are accepting *weak ordering* as a comparator, and were expanded with heterogeneous lookup (`is_transparent` nested typename).

To illustrate, lets us consider following example, we want to have mapping from the employee's name to the full object, the easiest approach would be to use the following map:

```
std::map<std::string, Employee> nameMap;
```

The drawback of above solution is that the name of the employee is held twice in the structure, once as a key, and once inside the object. To mitigate the problem, we could construct a `set` of `Employee` that compares only its name, i.e.

```
std::set<Employee, NameLess> nameSet;
```

In addition we can allow queering `nameSet` by actual names, by adding `is_transparent` nested typedef to the `NameLess` comparator. This solution esentially removes the value duplication.

Note that if the set definition would be changed, so the provided comparator is required to satisfy `StrictWeakOrdering` the code will no longer compile due to `CommonReference` requirement - this is same situation as in case of `equal_range` example presented in first paragraph. As a consequence, constraining the ordered containers with current `StrictWeakOrdering` concept, will de-facto remove functionality of heterogeneous lookup from ordered associative containers.

Of course, it would be possible to introduce a second concept that would be used for ordered containers, however creating multiple concepts representing the same mathematical concept (*weak ordering*) is against Design Ideas (section 1.3) presented in the The Palo Alto report: limiting number of concepts and providing concept representing general idea in the domain.

3.4. Hacking `common_reference`

It might be pointed out that the examples presented above as "not working", may be fixed by provding the `common_reference` for the compared types. As the compared types may be unrelated (one is subobject is the other), such specialization, will not have expected semantics of common type - once the `Employee` will be reduced to only its name, it cannot be reconstructed back.

To illustrate, lets as return to our `NameLess` example, presented at beginning of this section. For the code to compile, we need to make sure that the `CommonReference<Employee, std::string_view>` is satisfied - to achieve that we may provide an implicit conversion from `Employee` to `std::string_view` that would return `name` member.

However, this solution is riddled with problems. Not only it is introducing undesired conversion to the `std::string_view` that affects existing code, but fails in situations when we want to provide relation comparing other members of class `Employee` that have `std::string type`. For example, if we will want to write a second relation, `SurnameLess` that would compare the `surname` member of the `Employee` class, that also happen to have `std::string` type, we will also need to introduce conversion to `std::string_view` that would return `surname` member, which directly conflicts with conversion we have provided to support `NameLess`.

[Note: The conversion operator needs to return corresponding members of the `Employee`, to satisfy semantic requirements of the `Relation` concept, i.e:

- `NameLess{}(e, sv)` (`e.name < sv`), needs to be equivalent to `NameLess{}(std::string_view(e), sv)` (`std::string_view(e) < sv`),
- `SurnameLess{}(e, sv)` (`e.surname < sv`), needs to be equivalent to `SurameLess{}(std::string_view(e), sv)` (`std::string_view(e) < sv`).

Above requirements cannot be both satisfied at the same time, as they require conversion from `Employee` to `std::string_view` to have different semantics.]

We may attempt to mitigate need of implicit conversion, by providing an specialized common type for the `Employee` to `std::string_view`, for example:

```
class EmployeeNameView
{
  EmployeeNameView(std::string_view s) : value(s) {}
  EmployeeNameView(Employee const& e) : value(s.name) {}

  std::strong_ordering operator<=>(EmployeeNameView, EmployeeNameView) = default;

private:
  std::string_view value;
};
```

And then provide an explicit specialization of `common_type` (or `basic_common_reference`) for `Employee` and `std::string_view`:

```
namespace std
{
  template<>
```

```
  struct common_type<Employee, std::string_view>
  { using type = EmployeeNameView; };

  template<>
  struct common_type<std::string_view, Employee>
  { using type = EmployeeNameView; };
}
```

Again this solution fails, if we also want to introduce the `SurnameLess` relation. For these relation we need to specialize `common_type` for `Employee` and `std::string_view` to point to `EmployeeSurnameView` type, that is very similar to the `EmployeeNameView` with one important difference - the construction of this type from `Employee` object needs to store view to the `surname` member. This in the end leads to conflicting declaration of `common_type`, leading to same problem as in case of solution using conversion operator. Futhermore above, solutions would not applicable, if the `NameLess` would accept `std::shared_ptr<Employee>` (or any other library wrapper specialized for `Employee`), as we cannot specialize `common_type` or `common_reference` for types that are not user defined. Which means we cannot use it to allow lookups into `vector` of optionals or smart pointers.

As we need to provide different `common_type` specialization for comparison with either `name` and `surname`, the other solution is to use a different type to represent such object, so instead of using `std::string_view` to represent above values, we will use specialized object (like above `EmployeeNameView` and `EmployeeSurnameView`) when search for value. Instead of:

```
std::equal_range(ve.begin(), ve.end(), "Kowalski"sv, NameLess{});
```

we will write

```
std::equal_range(ve.begin(), ve.end(), EmployeeNameView("Kowalski"));
```

Note that we not longer need to provide the comparator object, as `Employee` are comparable with `EmployeeNameView`, due to existence of implicit conversion. Essentially, this solution replaces custom comparison functor (which can be created using lambda) with custom type.

Finally, we could drop any attempts to provide an semantically meaningful type for the `CommonReference` and define `common_type` to be a variant of the `std::variant<Employee, std::string_view>` (or `basic_common_type` to return `variant` of references). Of course this workaround cannot be applied in case of relation accepting standard-library types specialized for user-defined type. Furthermore, it unnecessarily complicates the definition of the comparator operators, as they need to able to compare variant object with correct semantics.

3.5. Implementing mathematical comparison

In contrast to previous sections, when we were unable to implement desired ordering (or implement it effectively), in this section we will discuss the example ordering, that cannot be implemented as a comparator, due existing specialization of `common_type`.

Let us consider implementing an `MathematicalLess` functor, that will compare class of integral types, according to mathematical meaning of this relation, i.e. `MathematicalLess{}(-1, 1u)` return `true`. We may implement it as follows:

```
struct MathematicalLess
{
  template<typename T, typename U>
  bool operator()(T t, U u) const
  {
     using UT = std::make_unsigned_t<T>;
     using UU = std::make_unsigned_t<U>;
     if constexpr (std::is_signed_v<T> == std::is_signed_v<U>) // types has same signedness
       return t < u;
     else if constexpr (std::is_signed_v<T>) // U is unsigned
       return t < 0 ? true : UT(t) < u;
     else  // T is unsigned
       return u < 0 ? false : t < UU(u);
  }
};
```

Now let us discuss if `StrictWeakOrdering<MathematicalLess, unsigned, int>`. Firstly, the `unsigned` and `int` types has a common type: `std::common_type_t<unsigned, int>` is `unsigned`. Secondly `MathematicalLess` can be invoked with any combination of objects of type either `int` or `unsigned`. This means that we are satisfying our syntactic requirements.

However, if we consider following semantic requirement that the invocation of relation `r(a, b)` is always equivalent to invocation `r(C(a), C(b))`, where `C` is common reference type for `a` and `b`. In case of our functor, that requires that `MathematicalLess{}(-1, 1u)`, is required to return the same value as `MathematicalLess{}(unsigned(-1), unsigned(1u))`, which is equal to `false`. However, by the design of `MathematicalLess` the result shall be true..

Note that this problem cannot be fixed without introducing opaque typedefs, as user is not allowed to change the common type for built-in types.

4. Design Decisions

4.1. Requiring homogeneous invocation

Even after removal of the `CommonReference` requirement for the `StrictWeakOrdering`, this concept will be more strict that the minimal STL1 requirements - the `StrictWeakOrdering<F, T, U>` requires `F` requires that `F` is both invocable with combination of types (`Predicate<F, T, U>` and `Predicate<F, U, T>`), but also two object of the same type (`Predicate<F, T, T>` and `Predicate<F, U, U>`).

To illustrate, if we consider and reduced an version of the `NameLess` comparator:

```
struct ReducedNameLess
{
   bool operator()(Employee const& lhs, std::string_view rhs) const
   { return lhs.name < rhs; }

   bool operator()(std::string_view lhs, Employee const& rhs) const
   { return lhs < rhs.name; }
};
```

The following invocation compiles and gives correct result:

```
std::equal_range(ve.begin(), ve.end(), "Kowalski"sv, ReducedNameLess{});
```

as implementation of `std::equal_range` does not need to invoke supplied comparator on two objects of same collection. However, even with the proposed relaxation of the `StrictWeakOrdering` concept, the range version of the above invocation, will still fail to compile:

```
std::ranges::equal_range(ve.begin(), ve.end(), "Kowalski"sv, ReducedNameLess{});
```

The above additional requirement are required to fulfill the goal of *soudness* of the design - we explicitly require that the requirements of the invoked algorithm could be checkable as the runtime. In case of the `equal_range(range, value, comp)`, we require that the:

1. `comp` model *weak ordering* over sum of elements in `range` and `value`,
2. `range` elements is sorted in order defined by the `comp`.

The second requirement can be checked by simply checking if `std::is_sorted(range.begin(), range.end(), comp)` is true, which impose that `comp` can be invoked with two elements of supplied range. For the first requirement we need to verify axioms described in section 3.2. Mathematical soundness of this paper - such validation requires `comp` to be invocable with any combination of types.

4.2. No relaxation for `StrictTotallyOrderedWith<T, U>`

The `StrictTotallyOrderedWith<T, U>` concept may be viewed as the special case of the `StrictWeakOrdering<std::less<>, T, U>`, where the ordering is represented as `operator<` instead of custom comparator. With such view we may want to apply similar relaxation of requirement for this concept.

However, the underlying semantics of the `StrictTotallyOrderedWith<T, U>` is different than the `StrictWeakOrdering<std::less<>, T, U>`, as it is meant to represent *strong order* - in contrast to the *weak ordering* for the *strong ordering*, if `a` and `b` are not ordered (`!(a < b) && !(b < a)`), then they are the same (`a == b`). In other words, even if situation when types of `a` and `b` differs, they are different representation of the same entity (e.g. `std::string` and `std::string_view`), and should have a notion of "common type" (e.g. sequences of characters).

Unfortunately, in some cases, implementing an representation of such common type, may be task that is orders of magnitude harder than implementing comparator itself. To illustrate lets as consider the comparison between class representing variable length integer `wide_int i` and floating point value `double d`. The implementation of such comparison may work as follow:

- represent integral part of `std::floor(d)` as `wide_int di`,
- if `i != di`, return `i < di`,
- otherwise, return `false`, if fraction part of `d` (`d - std::floor(d)`) is non-zero.

In contrast providing a `common_type<wide_int, double>` would require the user to implement class representing arbitrary precision floating point numbers.

Changes proposed in this paper allow to mitigate the problem, by implementing the above compassion custom comparison function. Note that without proposed relaxation of `StrictWeakOrdering` requirements, the burden of implementing such common type would be unavoidable.

5. Proposed Wording

The proposed wording changes refer to N4762 (C++ Working Draft, 2018-07-07).

Apply following changes to the [concept.relation] Concept `Relation` section:

```
template<class R, class T, class U>
concept Relation =
  Predicate<R, T, T> && Predicate<R, U, U> &&
  CommonReference<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
  Predicate<R,
    common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>,
    common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>> &&
  Predicate<R, T, U> && Predicate<R, U, T>;
```

Let

- r be an expression such that decltype((r)) is R;
- t be an expression such that decltype((t)) is T;
- u be an expression such that decltype((u)) is U; and
- C be common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>

~~Relation<R, T, U> is satisfied only if~~

- ~~`bool(r(t, u)) == bool(r(C(t), C(u)))`.~~
- ~~`bool(r(u, t)) == bool(r(C(u), C(t)))`.~~

Note that the definition of concept `StrictWeakOrder` in [concept.strictweakorder] does not need to be updated as it does not rely on the existence of `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>` in any way.

6. Acknowledgements

Casey Carter, Andrzej Krzemieński and Michał Łoś offered many useful suggestions and corrections to the proposal.

7. References

1. Alexander Stepanov, Paul McJones, "Elements of Programming", Addison-Wesley Professional; 1 edition (2009)
2. Bjarne Stroustrup et al., "A Concept Design for the STL", (N3351, https://wg21.link/n3351)
3. Eric Niebler, Casey Carter, Christopher Di Bella, "The One Ranges Proposal", (P0896R2, https://wg21.link/p0896r2)
4. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4762, https://wg21.link/n4762)